Practical 9

Aim: Python Django Framework – I

9.1: Introduction to MVC Framework, Django Installation, Server Configuration, ports, sample website development

1. Introduction to MVC Framework

MVC(Model-View-Controller)is a design pattern used in web development to separate an applicationinto three components.

1. **Model:** Manages application data and interacts with the database.
2. **View:** Displays data to the user through the user interface.
3. **Controller:** Handles user input and connects the Model and View.

2. Django Installation

DjangoisaPythonweb framework used to develop secure and dynamic websites.

Steps:

1. Install Python on the system.
2. Open Command Prompt.
3. Install Django using the followingcommand:
   pip install django
4. Verify theinstallation:
   python -m django –version

3. Project Creation

Create a new Django project using the command:

django-admin startproject projectname

Navigate to the project directory:

cd projectname

4. Server Configuration

Start the Djangodevelopment server:
python manage.py runserver
Open a web browser and visit:
http://127.0.0.1:8000/
The default Django welcome page will appear if the server starts successfully.

5. Ports

Portsidentify specific services running on a computer.

- **Port 80:** Default HTTP port.
- **Port 443:** Default HTTPS port.
- **Port 8000:** Default Django development server port.

To run Django on another port, use:

python manage.py runserver 8080

Then visit http://127.0.0.1:8080/.

6. My Website Development

Create an application:
python manage.py startapp practical91

```
D:\Kirtan_Django>venv\Scripts\activate
```

```
(venv) D:\Kirtan_Django>cd practical91

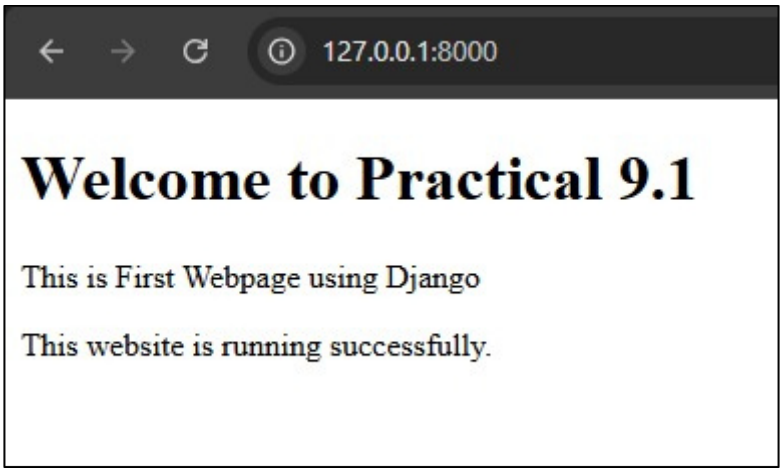
(venv) D:\Kirtan_Django\practical91>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  No migrations to apply.
```

```
(venv) D:\Kirtan_Django\practical91>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
October 18, 2026 - 21:40:58
Django version 6.1.2, using settings 'practical91.settings'
Starting WSGI development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/6.1/howto/deployment/
[18/Oct/2026 21:41:40] "GET / HTTP/1.1" 200 232
Not Found: /favicon.ico
[18/Oct/2026 21:41:43] "GET /favicon.ico HTTP/1.1" 404 2531
```

Output:



9.2: Django Template tags, variables, loops, control structures. Etc.

1. If Tag
The{%if%}tagdisplays content based on a condition.

```
{%ifuser.is_authenticated %}
   Welcome,{{user.username }}!
{% else %}
   Please log in.
{% endif %}
```

2. For Tag
The{%for%}tagisused to iterate over a list or other iterable objects.

```
{%foriteminitems%}
<p>{{ item }}</p>
{% endfor %}
```

3.BlockandExtendsTags
Thesetagsareusedfortemplate inheritance. The block tag defines a section, and the extends taginheritsanothertemplate.

Basetemplate(base.html):

```
<html>
<head>
   <title>{% block title %}My Website{% endblock %}</title>
</head>
<body>
   {% block content %}{% endblock %}
</body>
</html>
```

Child template (home.html):

```
{%extends "base.html" %}


{%block title %}Home Page{% endblock %}

{%block content %}
   <h1>Welcome to My Website</h1>
{%endblock %}
```

4.Include Tag
The{%include%} tag inserts another template into the current template.

```
{%include "header.html" %}
```

5. URL Tag
The {%url%} tag generates a URL using a named URL pattern.

```
<a href="{% url 'home' %}">Home</a>
```

6. CSRF Token Tag
The{% csrf_token%} tag provides protection against Cross-Site Request Forgery attacks in forms.

```
<form method="post">
{% csrf_token %}
<button type="submit">Submit</button>
</form>
```

7.Comment Tag
The {% comment %} tag is used to add comments that are not displayed in the final HTML output.

```
{% comment %}
This is a comment.
{% endcomment %}
```

8.With Tag
The{% with %} tag stores a value in a temporary variable within a limited scope.

```
{%with user.username as username %}
   Welcome, {{ username }}!
{%endwith %}
```

9.Now Tag
The{% now %} tag displays the current date and time in the specified format.

```
Today's date: {% now "Y-m-d H:i:s" %}
```

Variables in Django Templates
Variablesare used to displaydatapassed from a view to a template. They are written inside double curly braces {{ }}.

Example:

```
<h1>Welcome, {{ user.username }}!</h1>
<p>Product: {{ product.name }}</p>
```

1. Loops in Django Templates
Loops areusedtodisplaymultiple items from a list or database.

Example:

```
<ul>
{% for item in items %}
<li>{{ item }}</li>
{% endfor %}
</ul>
```

2. Control Structures

- If-Else Statement

It displays different content depending on whether a condition is true or false.

```
{% if user.is_authenticated %}

   Welcome, {{ user.username }}!
{% else %}
   Please log in.
{% endif %}
```

- If-Elif-Else Statement

It checks multiple conditions and displays the appropriate content.

```
{% if user.is_superuser %}
   You are an admin.
{% elif user.is_staff %}
   You are a staff member.
{% else %}
   You are a regular user.
{% endif %}
```

Create the Django Application

python manage.py startapp practical92

```
D:\Kirtan_Django>venv\Scripts\activate
```

```
(venv) D:\Kirtan_Django>cd practical92

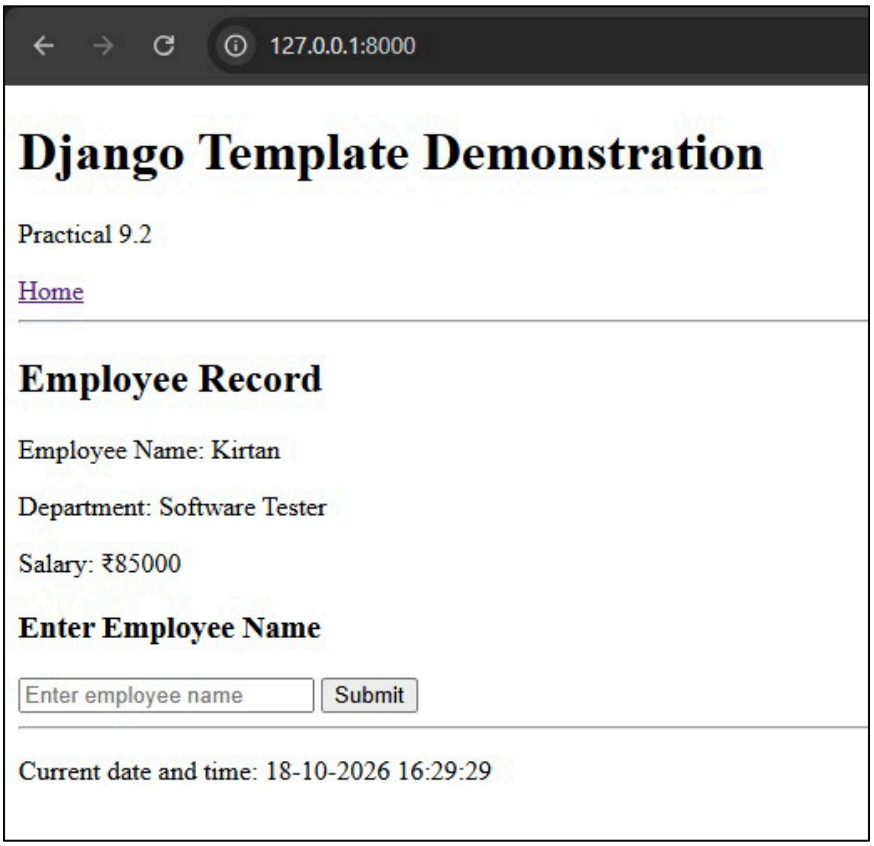
(venv) D:\Kirtan_Django\practical92>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  No migrations to apply.
```

```
(venv) D:\Kirtan_Django\practical92>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
October 18, 2026 - 21:55:23
Django version 6.1.2, using settings 'practical92.settings'
Starting WSGI development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting.
For more information on production servers see: https://docs.djangoproject.com
[18/Oct/2026 21:55:38] "GET / HTTP/1.1" 200 695
```

Output:


127.0.0.1:8000
Django Template Demonstration
Practical 9.2
Home
Employee Record
Employee Name: Kirtan
Department: Software Tester
Salary: ₹85000
Enter Employee Name
Enter employee name
Submit
Current date and time: 18-10-2026 16:29:29

9.3: Dynamic page that reads user data in a form and displays in another page.

Create a Django Project

```
django-admin startproject pr9_3
cd pr9_3
```

Register the Application

Open pr9_3/settings.py and add 'app9_3' to INSTALLED_APPS.

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'app9_3',
]
```

Definethe Model

File: app9_3/models.py

```
from django.db import models

class UserData(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField()

    def __str__(self):
        return self.name
```

Create a Form

Create a new file named forms.py inside the app9_3 folder.

File: app9_3/forms.py

```
from django import forms
from .models import UserData

class UserDataForm(forms.ModelForm):
    class Meta:
        model = UserData
        fields = ['name', 'email']
```

Create Views

File: app9_3/views.py

```
from django. shortcuts import render, redirect
from .forms import UserDataForm
from .models import UserData

def user_data_input(request):
    if request.method == 'POST':
        form = UserDataForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('display_data')
    else:
        form = UserDataForm()

    return render(request, 'user_data_input.html', {'form': form})

def display_user_data(request):
    user_data = UserData.objects.all()
    return render(request, 'display_data.html', {'user_data': user_data})
```

Create HTML Templates

Create a folder named templates inside app9_3.
Create the following two HTML files inside it.

File 1: user_data_input.html

```
<!DOCTYPE html>
<html>
<head>
    <title>User Data Input</title>
</head>
<body>
    <h2>User Data Input</h2>

    <form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Submit</button>
    </form>
</body>
</html>
```

display_data.html

```
<!DOCTYPE  html>
<html>
<head>
   <title>Display User Data</title>
</head>
<body>
   <h2>Display User Data</h2>

   <table border="1" cellpadding="8">
      <tr>
         <th>Name</th>
         <th>Email</th>
      </tr>

      {% for user in user_data %}
      <tr>
         <td>{{  user.name  }}</td>
         <td>{{ user.email }}</td>
      </tr>
      {% empty %}
      <tr>
         <td colspan="2">No user data available.</td>
      </tr>
      {% endfor %}
   </table>

   <p><a href="{% url 'input_data' %}">Back to Input Page</a></p>
</body>
</html>
```

Configure Application URLs

```
Create app9_3/urls.py.
from django.urls import path
from . import views

urlpatterns = [
   path('input/', views.user_data_input, name='input_data'),
   path('display/', views.display_user_data, name='display_data'),
]
```

Run the Development Server

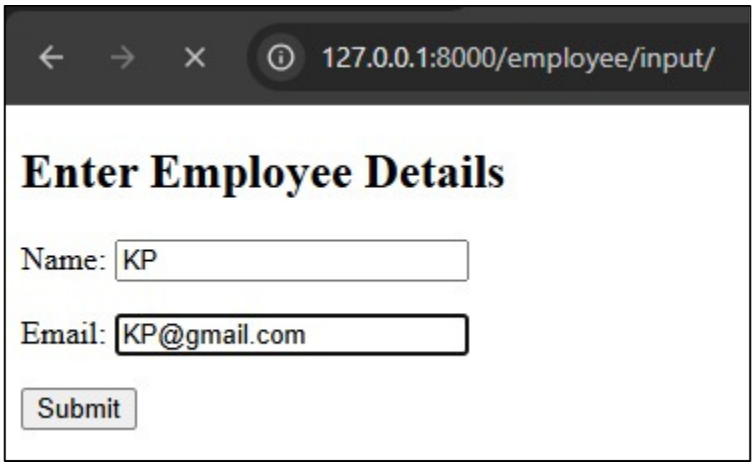
```
D:\Kirtan_Django>venv\Scripts\activate

(venv) D:\Kirtan_Django>cd practical93
```

```
(venv) D:\Kirtan_Django\practical93>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
October 18, 2026 - 23:17:57
Django version 6.1.2, using settings 'practical93.settings'
Starting WSGI development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Output: (http://127.0.0.1:8000/employee/input/)



(http://127.0.0.1:8000/employee/display/)



9.4: Upload and download files from the server.

Activate the Virtual Environment

```
Kirtan_Django\Scripts\activate.bat
```

Create a Django Project

```
django-admin startproject practical94
cd practical94
```

Create an Application

```
python manage.py startapp file_manager
```

Configure Project Settings

Open settings.py.

```
Add 'file_manager' to the existing INSTALLED_APPS list.
INSTALLED_APPS = [
    #       Existing       applications
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'file_manager',
]
```

Create a Model for Files

file_manager/models.py

```
from django.dbimportmodels

class File(models.Model):
    name = models.CharField(max_length=255)
    file = models.FileField(upload_to='uploads/')
    uploaded_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.name
```

Create a Form for File Upload

```
Create file_manager/forms.py.
fromdjango import forms
from.models import File

classFileUploadForm(forms.ModelForm):
    class Meta:
        model = File
        fields = ('name', 'file')
```

CreateViews for Uploading and Downloading Files

file_manager/views.py

```
fromdjango.shortcuts import render, redirect, get_object_or_404
fromdjango.http import FileResponse
from.modelsimport File
from.formsimport FileUploadForm

defupload_file(request):
    ifrequest.method == 'POST':
        form=FileUploadForm(request.POST, request.FILES)
        ifform.is_valid():
            form.save()
            returnredirect('file_list')
    else:
        form=FileUploadForm()

    returnrender(request, 'file_manager/upload_file.html',
                 {'form': form})

deffile_list(request):
    files=File.objects.all()
    returnrender(request, 'file_manager/file_list.html',
                 {'files': files})

defdownload_file(request, file_id):
    file_obj=get_object_or_404(File, pk=file_id)
    returnFileResponse(
        file_obj.file.open('rb'),
        as_attachment=True,
        filename=file_obj.name
    )
```

Create HTML Templates

file_manager/templates/file_manager/
Also createbase.htmlinsidefile_manager/templates/.

File 1: base.html
Path: file_manager/templates/base.html

```
<!DOCTYPE html>
<html>
<head>
   <title>File Manager</title>
</head>
<body>
   <h1>Django File Manager</h1>
   <hr>
   {% block content %}{% endblock %}
</body>
</html>
```

File 2: upload_file.html
Path:file_manager/templates/file_manager/upload_file.html

```
{% extends "base.html" %}
{% block content %}
<h2>Upload File</h2>
<form method="post" enctype="multipart/form-data">
   {% csrf_token %}
   {{ form.as_p }}
   <button type="submit">Upload</button>
</form>

<p><a href="{% url 'file_list' %}">Back to File List</a></p>
{% endblock %}
```

File 3: file_list.html
Path: file_manager/templates/file_manager/file_list.html

```
{% extends "base.html" %}
{% block content %}
<h2>File List</h2>

<ul>
   {% for file in files %}
      <li>
         {{ file.name }}
         <a href="{% url 'download_file' file.id %}">
            Download
         </a>
      </li>
   {% empty %}
      <li>No files have been uploaded yet.</li>
   {% endfor %}
</ul>
<p><a href="{% url 'upload_file' %}">Upload File</a></p>
{% endblock %}
```

Configure Application URLs

Create file_manager/urls.py.

```
fromdjango.urls import path
from.import views

urlpatterns = [

    path('upload/', views.upload_file, name='upload_file'),
    path('download/<int:file_id>/', views.download_file,
        name='download_file'),
    path('files/', views.file_list, name='file_list'),
]
```

Include Application URLs

```
File: practical94/urls.py
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('file_manager/', include('file_manager.urls')),
]
```

Run this Application

```
D:\Kirtan_Django>venv\Scripts\activate

(venv) D:\Kirtan_Django>cd practical94
```

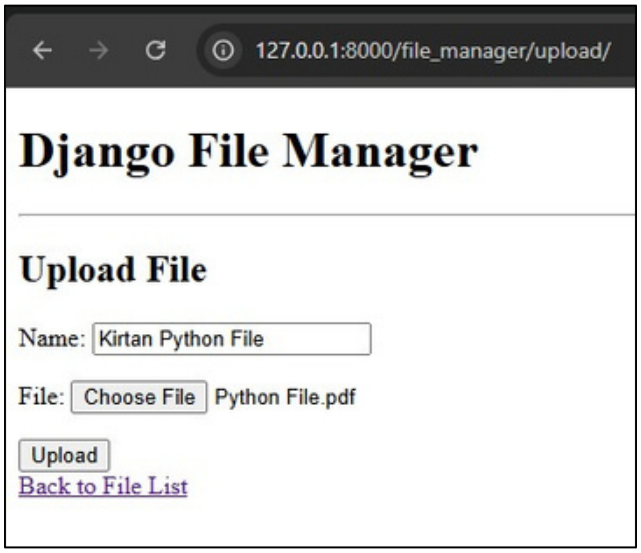
```
(venv) D:\Kirtan_Django\practical94>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, file_manager, sessions
Running migrations:
  No migrations to apply.

(venv) D:\Kirtan_Django\practical94>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

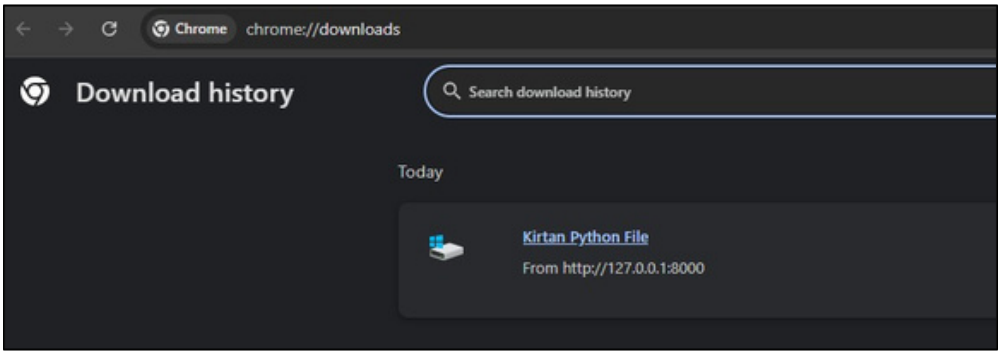
System check identified no issues (0 silenced).
October 19, 2026 - 11:48:50
Django version 6.1.2, using settings 'pr9_4.settings'
Starting WSGI development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting.
For more information on production servers see: https://docs.djangoproject.co
Not Found: /
[19/Oct/2026 11:49:06] "GET / HTTP/1.1" 404 2384
```

Output:


127.0.0.1:8000/file_manager/upload/
Django File Manager
Upload File
Name: Kirtan Python File
File: Choose File Python File.pdf
Upload
Back to File List


127.0.0.1:8000/file_manager/files/
Django File Manager
File List
Kitu Download
Kirtan Python File Download
Upload File


Chrome
chrome://downloads
Download history
Search download history
Today
Kirtan Python File
From http://127.0.0.1:8000

9.5: User Authentication by comparing valid user name and password from the database using PostgreSQL database.

Create a Django Project

OpenCommandPrompt and execute:

```
django-admin  startproject  practical95
cd practical95
```

Create an Application

```
python manage.py startapp authentication_app
```

Install PostgreSQL Support

Install the PostgreSQL adapter inside your activated virtual environment:
python -m pip install "psycopg[binary]"
Make sure PostgreSQL is installed and its database service is running.

Configure PostgreSQL

Open practical95/settings.py and configure the database settings.

```
DATABASES = {
   'default': {
      'ENGINE': 'django.db.backends.postgresql',
      'NAME': 'postgres',
      'USER': 'postgres',
      'PASSWORD': kitu6681',
      'HOST': 'localhost',
      'PORT': '5432',
   }
}
```

Add 'authentication_app' to the existing INSTALLED_APPS list.

```
INSTALLED_APPS = [
   'django.contrib.admin',
   'django.contrib.auth',
   'django.contrib.contenttypes',
   'django.contrib.sessions',
   'django.contrib.messages',
   'django.contrib.staticfiles',
   'authentication_app',
]
```

Configure the custom user model and authentication backend:

```
AUTH_USER_MODEL = 'authentication_app.CustomUser'

AUTHENTICATION_BACKENDS = [
   'django.contrib.auth.backends.ModelBackend',
]
```

Create a Custom User Model

```
File: authentication_app/models.py
from django.contrib.auth.models import (
    AbstractBaseUser,
    BaseUserManager,
    PermissionsMixin
)
fromdjango.db import models
fromdjango.utils import timezone


classCustomUserManager(BaseUserManager):
    defcreate_user(self, username, password=None, **extra_fields):
        ifnot username:
            raise ValueError('Username must be provided')

        user = self.model(username=username, **extra_fields)
        user.set_password(password)
        user.save(using=self._db)
        return user

    defcreate_superuser(self, username, password=None, **extra_fields):
        extra_fields.setdefault('is_staff', True)
        extra_fields.setdefault('is_superuser', True)

        return self.create_user(username, password, **extra_fields)


classCustomUser(AbstractBaseUser, PermissionsMixin):
    username = models.CharField(max_length=255, unique=True)
    is_active = models.BooleanField(default=True)
    is_staff = models.BooleanField(default=False)
    date_joined = models.DateTimeField(default=timezone.now)

    objects = CustomUserManager()

    USERNAME_FIELD = 'username'

    def__str__(self):
        return self.username
```

Create Authentication Views

```
File: authentication_app/views.py
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.decorators import login_required
from django.shortcuts import render, redirect
```

```
deflogin_view(request):
    error = ''

    ifrequest.method == 'POST':
        username=request.POST.get('username', '')
        password=request.POST.get('password', '')

        user=authenticate(
            request,
            username=username,
            password=password
        )

        ifuserisnotNone:
            login(request, user)
            returnredirect('success')
        else:
            error='Invalid username or password.'

    returnrender(request, 'login.html', {'error': error})


@login_required
defsuccess_view(request):
    returnrender(request, 'success.html')


deflogout_view(request):
    logout(request)
    returnredirect('login')
```

CreateHTMLTemplates

authentication_app/templates/

File 1:login.html

```
<!DOCTYPE  html>
<html>
<head>
    <title>Login</title>
</head>
<body>
    <h2>Login</h2>

    {% if error %}
        <p>{{ error }}</p>
    {% endif %}

    <form method="post">
    {% csrf_token %}

    <label>Username:</label>
    <input type="text" name="username" required>
```

```
        <br><br>

        <label>Password:</label>
        <input type="password" name="password" required>
        <br><br>

        <button type="submit">Login</button>
    </form>
</body>
</html>
```

File 2: success.html

```
<!DOCTYPE  html>
<html>
<head>
    <title>Login Successful</title>
</head>
<body>
    <h2>Login Successful!</h2>
    <p>Welcome, {{ request.user.username }}.</p>

    <a href="{% url 'logout' %}">Logout</a>
</body>
</html>
```

ConfigureApplicationURLs

Create authentication_app/urls.py.

```
from django.urls import path
from . import views


urlpatterns = [
    path('login/', views.login_view, name='login'),
    path('success/', views.success_view, name='success'),
    path('logout/', views.logout_view, name='logout'),
]
```

IncludeAuthentication URLs
File: practical95/urls.py

```
from django.contrib import admin
from django.urls import path, include


urlpatterns = [
    path('admin/', admin.site.urls),
    path('auth/', include('authentication_app.urls')),
]
```

Run this Practical

```
D:\Kirtan_Django>venv\Scripts\activate
```

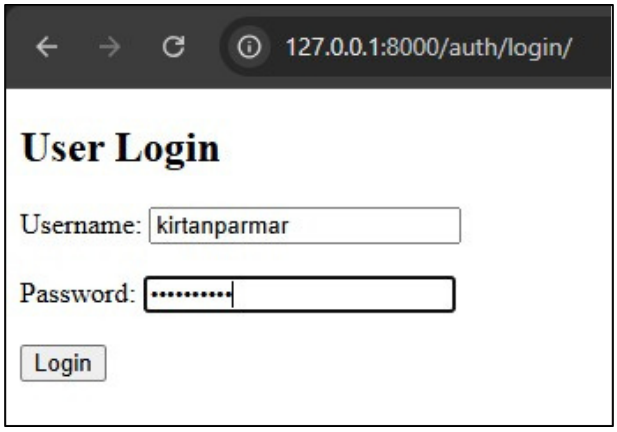
```
(venv) D:\Kirtan_Django>cd practical95
```

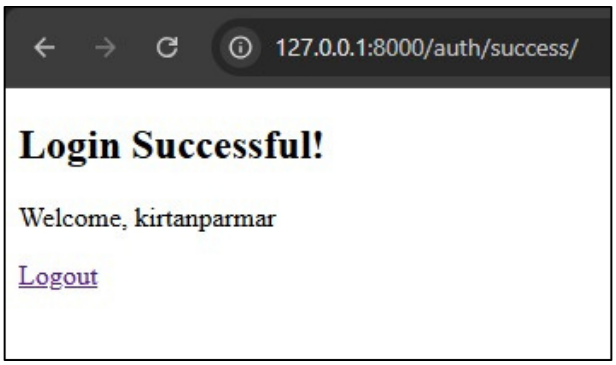
```
(venv) D:\Kirtan_Django\practical95>python manage.py createsuperuser
Username: kirtanparmar
Password:
Password (again):
Superuser created successfully.
```

```
(venv) D:\Kirtan_Django\practical95>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
October 19, 2026 - 12:24:48
Django version 6.1.2, using settings 'pr9_5.settings'
Starting WSGI development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Output:





Practical 10

Aim: Create REST API using Python Flask to send the form data like username, email into the database and test that API.

Install Required Libraries

Open Command Prompt and execute:
pip install Flask Flask-SQLAlchemy

Create Flask Application

Create a file named app.py and write the following code:

```
from flask import Flask, request, jsonify
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)

# Configure SQLite database
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///mydatabase.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

db = SQLAlchemy(app)

# Create User model
class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)

# Create database table
with app.app_context():
    db.create_all()

# API to receive and store user data
@app.route('/api/user', methods=['POST'])
def create_user():
    data = request.get_json(silent=True) or {}

    username = data.get('username')
    email = data.get('email')

    if not username or not email:
        return jsonify({
            'message': 'Username and email are required'
        }), 400

    user = User(username=username, email=email)

    try:
        db.session.add(user)
        db.session.commit()
```

```
    exceptException:
        db.session.rollback()
        returnjsonify({
            'message': 'Username or email already exists'
        }), 409

    returnjsonify({
        'message': 'User created successfully',
        'username': username,
        'email':email
    }), 201

if__name__== '__main__':
    app.run(debug=True)
```

Run theFlaskApplication

Open Command Prompt in the folder containing app.py and execute:
python app.py
Expected output:
* Running on http://127.0.0.1:5000

Test the API Using Postman

1. Open Postman.
2. Select the POST method.
3. Enter the URL: http://127.0.0.1:5000/api/user
4. Select Body → raw → JSON.
5. Enter the following data:

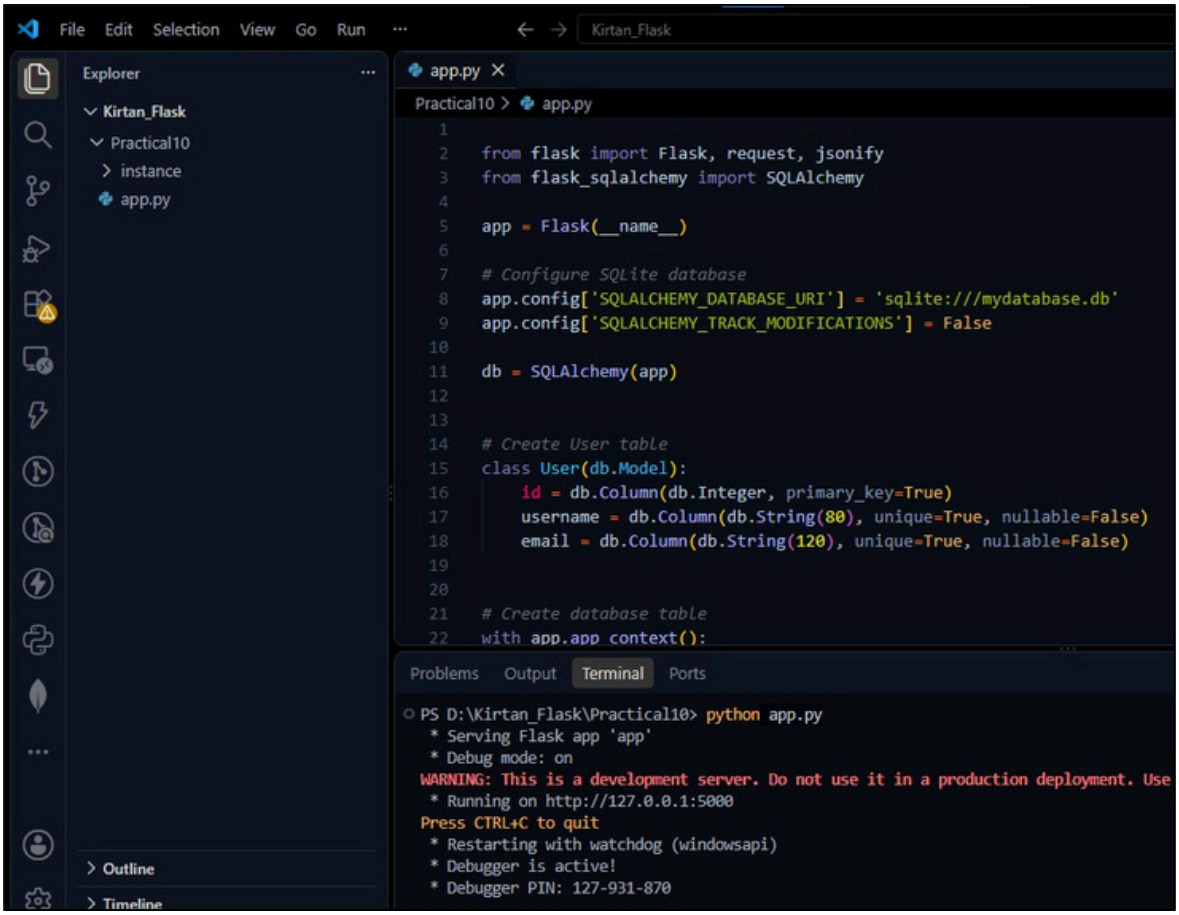
```
{
    "username": "kirtanparmar",
    "email": "k@gmail.com"
}
```

Click Send.

Expected response:
Status: 201 Created

```
{
    "message": "User created successfully",
    "username": "kirtanparmar",
    "email": "k@gmail.com"
}
```

Run this:



Output:

